



Gymnázium, Praha 6, Arabská 14
předmět programování, vyučující Ing. Daniel Kahoun

Simulátor stolní karetní hry pro více hráčů na síti

ročníkový projekt

ROČNÍKOVÁ PRÁCE

Předmět: Programování

Téma: Simulátor stolní karetní hry pro více hráčů
na síti

Autor: Jan Hlavnička

Třída: 2.F

Školní rok: 2025/2026

Vedoucí práce: Ing. Daniel Kahoun

Třídní učitel: RNDr. Jana Halenková

Anotace

Tato ročníková práce se zabývá návrhem a implementací univerzálního herního enginu pro karetní hry v prostředí webového prohlížeče. Jádro systému tvoří backendová aplikace naprogramovaná v jazyce Java, která využívá architekturu klient-server a zajišťuje plynulou asynchronní komunikaci v reálném čase prostřednictvím protokolu WebSocket. Výrazným technologickým prvkem architektury je její modularita – systém umožňuje dynamické načítání nezávislých herních modulů (pluginů) pomocí vlastního SDK, aniž by bylo nutné zasahovat do zdrojového kódu samotného serveru.

Klientská část je realizována jako moderní interaktivní webová aplikace s využitím frameworku React a jazyka TypeScript. Pro ovládání používá primárně metody Drag & Drop a systému dialogů. Součástí projektu je rovněž praktická implementace pravidel populárních karetních her Bang!, Prší a UNO, na kterých byla stabilita, ochrana proti neplatným tahům a plynulost enginu úspěšně otestována v reálném síťovém provozu. Výsledkem práce je plně funkční a nasazená platforma, připravená pro budoucí rozšiřování.

Poděkování

Chtěl bych poděkovat především mému vedoucímu práce ing. Danielu Kahounovi za vedení práce a vstřícnost při konzultacích. Dále bych chtěl poděkovat své rodině a kamarádům za to, že mojí hru mnohokrát vyzkoušeli a upozornili na její nedostatky.

Prohlášení

Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu dle těchto podmínek: U textové dokumentace se toto oprávnění uděluje pro účely archivace a prezentace školy. Nakládání se zdrojovým kódem a dalšími softwarovými komponentami se řídí pouze příloženou licencí v repozitáři projektu. Na obrázky ve složce public se license ani prohlášení nevztahují – nejsou ani součástí GitHub repozitáře.

Text dokumentace i kód byly vytvořeny výhradně autorem. Umělá inteligence byla použita jako užitečný nástroj pro kontrolu gramatiky, automatickému doplňování kódu, vygenerování dvou obrázků postav a jednoduchou pomoc při tvorbě, to ovšem pouze ve velmi malém rozsahu a s dohledem na její výstup. Ostatní případné využití jsou na daných místech uvedena.

V Praze dne _____

Jan Hlavnička

Obsah

1	Úvod	6
2	Cíle práce	6
3	Předem stanovená struktura práce a práce se zdroji	7
4	O hře Bang!	7
4.1	Pravidla hry	8
4.1.1	Příprava hry	8
4.1.2	Cíl hry	8
4.1.3	Průběh kola	8
4.2	Průběh hry	8
5	Vyzkoušení hry	8
6	Návod k použití	8
7	Technologie	9
8	Implementace serveru	9
8.1	KomunikátorHry a SocketServer	10
8.2	Interface pro balíčky HerniPlugin a HerniPravidla	10
8.3	Karta	11
8.3.1	HratelnaKarta	11
8.3.2	VylozitelnaKarta	11
8.4	NacitacPluginu a SpravceHernichPravidel	11
8.5	Hra a HraImp	12
8.6	Spravce tahu	13
8.7	Hrac a HracImp	14
8.7.1	boolean odeberZivot() a boolean pridejZivot()	14
8.7.2	Akce s kartou	14
8.7.3	vzdalenostPod(int max, boolean zpětne)	14
9	Implementace pluginů	15
9.1	Bang!	15
9.2	Prší	16
9.3	Uno	16
9.4	Budoucí pluginy	16
10	Protokol	17
11	Implementace klienta	18
11.1	O aplikaci	18
11.2	Hlavní objekt gamestate	18
11.3	gameActions.ts	18
11.4	Připojovací stránka LoginPage	19
11.5	Herní stránka GamePage	19
11.6	Drag & drop	19
11.7	Notifikace	19
11.8	Dialog	20
11.9	Tmavý režim	20
11.10	Server info a game info	20
11.11	Původní klient	20
12	Utility pro vývoj	21
13	Build, hosting a nasazení	21
14	Testování	21
15	Řešené problémy	21

16 Budoucí rozšiřování	23
17 Závěr	23
18 Přílohy	24

1 Úvod

Společné hraní karetních a deskových her představuje oblíbený způsob trávení volného času pro řadu lidí. Poprvé mě myšlenka na naprogramování online karetní hry napadla již před několika lety. Tehdejší první pokusy však brzy ukázaly, že jsem se svými tehdejšími znalostmi nebyl schopen takovou hru vytvořit.

Impulzem k oživení tohoto nápadu se stalo hraní her o prázdninách, kdy mě napadlo, že by bylo dobré si moci hry zahrát, i když se zrovna nebudeme moci potkat osobně. Proto jsem se rozhodl naprogramovat *Bang!*, což díky již nabytým znalostem Javy z hodin programování a tvoření jiných projektů bylo možné.

Nejprve měl engine sloužit pouze pro *Bang!*, ale postupem času se ukázalo, že je zbytečné se soustředit pouze na jednu hru, když většina logiky a funkcí lze využít i pro jiné hry. Proto jsem se již v rané fázi rozhodl, že projekt bude zaměřený na univerzální herní engine pro různé karetní hry, přičemž *Bang!* bude pouze jedním z implementovaných pluginů.

Tato práce popisuje celý životní cyklus vývoje takového systému od prvotního návrhu až po jeho nasazení na server. První kapitoly se věnují specifikaci cílů a stručnému popisu pravidel implementovaných her. Následně práce detailně rozebírá architekturu celého systému založenou na modelu klient-server. V backendové části se zaměřuje na jazyk Java, komunikaci přes websocketový protokol a dynamické načítání nezávislých herních pluginů. Ve frontendové části je pak popsána tvorba interaktivního uživatelského rozhraní pomocí moderního frameworku React.

Výsledkem projektu je platforma, která hráčům umožňuje plynulý herní zážitek přímo ve webovém prohlížeči a případným vývojářům nabízí snadnou cestu k implementaci vlastních herních pravidel.

2 Cíle práce

Cíle práce byly definovány hned na začátku projektu. Byly zvoleny tak, aby obsahovaly původní hlavní cíl – vyrobit online *Bang!* – ale aby se zaměřily primárně na možnost vytvoření celého herního enginu, nejenom jednoduché hry. Cíle byly formulovány takto:

- Vytvořit síťovou karetní hru s architekturou klient-server, kde server bude zajišťovat logiku hry a klient bude sloužit k interakci hráče.
- Naprogramovat server v jazyce Java, který bude mimo jiné:
 - spravovat herní logiku,
 - kontrolovat platnost akcí hráčů,
 - zajišťovat průběh hry a aplikaci pravidel,
 - komunikovat s klienty pomocí websocketového protokolu.
- Implementovat webového klienta, který poběží v prohlížeči a umožní hráčům:
 - zobrazit karty v ruce a na stole,
 - sledovat stav a informace o ostatních hráčích,
 - provádět akce a ovládat hru prostřednictvím uživatelského rozhraní,
 - a další potřebné informace pro průběh hry.
- Podporovat variaci karetní hry *Bang!* s možností rozšíření o další hry.
- Otestovat aplikaci s reálnými hráči a vyhodnotit její funkčnost i uživatelskou přívětivost.
- Vypracovat dokumentaci a návod k použití, včetně popisu architektury a zdrojového kódu.

3 Předem stanovená struktura práce a práce se zdroji

Již na začátku školního roku 2025-26 bylo potřeba stanovit si osnovu, průběh práce a deklarovat svoji práci se zdroji. Byly zvoleny takto:

Průběh zpracování práce

1. vytvoření jednoduchého serveru
2. vytvoření jednoduchého klienta
3. přidání logiky a ovládacích prvků
4. testování hry s dalšími lidmi
5. opravení nedostatků na základě získaných podnětů
6. sepsání dokumentace a průběhu práce
7. přidání dodatečných méně potřebných funkcí
8. publikace práce

Osnova práce

- o hře Bang! (popřípadě jiných implementovaných hrách)
- návod k programu
- průběh tvorby
- dokumentace
- závěr

Práce se zdroji

K práci budu používat oficiální pravidla hry, dokumentaci Javy (docs.oracle.com), dokumentaci Reactu (react.dev) a stránky [w3schools.com](https://www.w3schools.com). Dále pro řešení problémů využiji internetových webových stránek, jako jsou například Stack Overflow, nástrojů umělé inteligence (ChatGPT, Copilot) a rad od lidí, kteří mají s problémem zkušenost, včetně vedoucího práce pana Ing. Daniela Kahouna.

4 O hře Bang!

Bang! je karetní hra s tematikou Divokého západu, jejímž autorem je italský herní designér Emiliano Sciarra. Od svého uvedení na trh v roce 2002 se hra zaměřuje na herní mechanismus skrytých rolí a postupné eliminace hráčů. Účastníci jsou rozděleni do tří skupin s protichůdnými cíli (šerif a jeho pomocníci, bandité, odpadlíci), přičemž identita většiny hráčů zůstává až do jejich vyřazení utajena. Hra kombinuje správu karet v ruce s prvkem náhody a odhadu ostatních.

V České republice patří Bang! k dlouhodobě nejprodávanějším titulům v kategorii karetních her. Jeho pozice na trhu je stabilní, což dokládá i široká dostupnost v běžné obchodní síti i specializovaných prodejnách. Výhradním distributorem a vydavatelem české verze je společnost Albi, která zajišťuje nejen prodej základní sady, ale i ucelené řady tematických rozšíření a variant. [33].

4.1 Pravidla hry

Hra implementuje upravenou verzi pravidel. Ta bude v této kapitole popsána. Celá originální pravidla lze najít na webu vydavatelství [Albi](#).

4.1.1 Příprava hry

Na začátku hry si každý hráč vybere jednu ze dvou vylosovaných postav. Tato postava mu zůstane po celou dobu hry. Postava má svůj efekt a počet životů. Hráč také dostane roli, kterou (pokud není šerif) ostatní nevidí. Každému hráči bude podle postavy přidělen počet karet.

Postava určuje hráčovu speciální schopnost, zatímco role určí jeho cíl hry.

4.1.2 Cíl hry

Cíl hry má každý napsaný na své roli – šerif s pomocníky musí zabít bandity, bandité šerifa a odpadlíky všechny hráče. Hráč umírá tím, že mu dojdou životy a nemá žádné *pivo*. Bandité mohou šerifa poznat podle hvězdy u jeho jména.

4.1.3 Průběh kola

Začíná šerif a hráči se střídají v pořadí, ve kterém se připojili. Pro hráče kolo začíná získáním dvou karet. V průběhu kola může hráč spálit, zahrát a vyložit libovolný počet karet. Výjimkou je karta *Bang*, kterou lze zahrát pouze jednou za tah. Tah hráč ukončí zmáčknutím tlačítka **ukončit tah**. Na konci tahu musí mít hráč v ruce tolik karet, kolik má maximálně životů.

4.2 Průběh hry

Hráči se snaží zbavit své nepřátele všech životů a zároveň sami neumřít. K tomu používají karty, které mohou zahrát nebo vyložit. Karty mají různé efekty, například mohou způsobit ztrátu životů, získání karet, ochranu před útoky a podobně. Většina na sobě má napsáno, jak fungují. Pokud karta pro pokračování hry vyžaduje prvek náhody, zajistí ho kolo štěstí. Karta *pivo* nemá žádný efekt, když zbývají pouze dva hráči.

5 Vyzkoušení hry

Kompletní hra včetně klienta, serveru a všech pluginů zmíněných v této práci je dostupná na webu bang.honzaa.cz. Zdrojové soubory projektu jsou umístěny v GitHub repozitáři [HonzaHlavnicka/Bang](#), kde také jdou přidávat *Issues* pro nalezené problémy.

6 Návod k použití

Chcete-li si zahrát hru, navštivte webovou stránku bang.honzaa.cz. Vyberte kartu **Vytvořit hru**, vyberte, jakou hru chcete hrát, zadejte své jméno a klikněte na tlačítko **Vytvořit**. Poté se připojíte k vytvořené hře a zobrazí se vám kód hry. Tento kód pošlete svým přátelům, aby se mohli také připojit. Ostatní hráči se připojí zadáním kódu v kartě **Připojit se ke hře**. Jakmile se připojí všichni hráči, klikněte na tlačítko **Spustit hru** a můžete začít hrát. Pokud někomu v průběhu hry vypadne spojení, může se do hry vrátit v sekci **Vrátit se**.

Na hracím poli vidíte dole své karty, nad kterými (popřípadě na menší obrazovce vlevo od nich) mohou být vyložené karty. Vlevo dole uvidíte svoji postavu, roli, své jméno a podobně. Nahoře uvidíte informace o ostatních hráčích, například jejich jména, postavy, role a podobně. Uprostřed jsou karty na stole a další herní prvky. Jmenovka hráče, která je zrovna žlutá, signalizuje, že je tento hráč na tahu.

Pokud jste na tahu, tak můžete podle pravidel hry, kterou hrajete, provádět různé akce. Chcete-li si líznout kartu, klikněte na balíček karet. Chcete-li zahrát kartu, tak ji pomocí myši nebo prstu přetáhněte na odhazovací balíček. Pokud chcete kartu spálit (tzn. odhodit bez efektu), tak ji přetáhněte na oheň vedle odhazovacího balíčku. Chcete-li kartu vyložit, tak ji přetáhněte na plochu nad svými kartami. Pokud chcete kartu vyložit před jiného hráče, tak ji stačí přetáhnout na něj. Pokud chcete ukončit tah, klikněte na tlačítko **Ukončit tah**. V závislosti na hře, kterou hrajete, mohou být některé prvky skryté.

Pokusíte-li se udělat něco, co nemůžete, například zahrát kartu mimo svůj tah, tak se vám vpravo nahoře ukáže upozornění, které vám řekne, co se stalo.

Hra vám může ukázat dialog, který od vás vyžaduje nějakou akci, například u *svrška* v Prší na jakou barvu chcete změnit. Tyto dialogy se zobrazují uprostřed obrazovky a obsahují potřebné informace a možnosti výběru. Některé z těchto dialogů lze zavřít kliknutím do volného prostoru mimo dialog. Pokud zrušíte dialog pro detaily karty, tak se vám karta nevrátí.

7 Technologie

Na engine hry se používá *Java 11*, která byla vybraná, aby mohl server běžet i na školním serveru, který novější nepodporoval. Java byla zvolena z důvodu, že je na Gymnáziu Arabská vyučována a protože z programovacích jazyků, které autor zná, byla pro projekt nejvhodnější. Knihovny na serveru, které se používají, jsou převážně *Java websocket*, která řeší serverovou logiku pro připojení k websocketu a vytvoření websocketového serveru. Dále se používá *org.json* pro práci s *JSON* objekty.

Na straně klienta se používá *React* s *Vite* a *TypeScriptem*. Nejdříve se používal *vanilla JavaScript*, který byl ovšem nevhodný kvůli těžší rozšiřitelnosti, modularitě a chybějící typové kontrole. Tato původní verze se v repozitáři stále nachází ve složce */klient*. Z frameworků byl zvolen právě *React*, protože je pro tento typ projektu vhodný a poskytoval dostatečné funkce i moduly. Z externích knihoven se používá *react-dnd* pro implementaci drag & drop logiky, *react-hot-toast* pro zobrazení upozornění a *react-custom-roulette* pro implementaci kola štěstí. Tyto moduly byly zvoleny, protože poskytují obsah, který není přímou součástí problematiky hry, a umožňují zaměřit se na důležitější problémy bez znovuvynalézání kola.

8 Implementace serveru

Server se dělí do několika balíčků a projektů. Hlavní balíček je **bang**, který propojuje **sdk** se **serverem**. Sdk je klíčová součást celého projektu, protože umožňuje pluginům nebýt závislými na konkrétní implementaci serveru a pracovat jen s interface. Balíček **server** je poté konkrétní implementací serverových tříd.

Plná dokumentace sdk je dostupná v javadoc dokumentaci dostupné na bang.honzaa.cz/apidocs. Tam jsou popsány všechny třídy, metody a proměnné, které jsou dostupné pro pluginy a pro serverové třídy. Tato kapitola je zaměřena na ty nejdůležitější a nejzajímavější části, které jsou klíčové pro pochopení celého systému.

8.1 KomunikátorHry a SocketServer

Pro komunikaci s klienty se používá websocket. Pro jeho správu v Javě se používá knihovna *Java websocket*. Díky této knihovně stačí dědit třídu `WebSocketServer` a implementovat metody jejích akcí. To dělá právě třída `SocketServer`. Třída obsluhuje zachycení výjimek, vytvoření her, načítání pluginů a komunikaci s hráči na mimoherní úrovni. Tou je myšleno stahování informací o instalovaných pluginech, získávání informací o serveru a podobně.

Nejdůležitější metodou je `onMessage`, která zpracovává příslušné zprávy. Dokumentace celého protokolu a všech zpráv je dostupná v dokumentech na [GitHubu](#). Tato metoda umí vytvářet hry a spojovat zprávy s příslušnými komunikátoryHer, které si drží v mapě `komunikatoryHracu`.

Třída také obsahuje logiku kontroly administrátorského hesla, včetně ochrany proti brute-force útokům.

`KomunikatorHry` je třída, která spojuje `SocketServer` a samotnou hru. Drží si seznamy hráčů, tokeny hráčů pro připojení, mapy hráčů se sockety a podobně. Je používána k tomu, aby mohly serverové nebo pluginové metody komunikovat s klientem. Nemusí k tomu znát přímo `WebSocket` objekt, protože ten si spojí v metodě `posli()` sám komunikátor. Komunikátor také zpracovává události posílané klientem přes `SocketServer`. Tyto události rovnou předává do metod hráče, kterého se týkají. Pro oddělení protokolu od samotné hry a pluginů komunikátor obsahuje konkrétní metody na odeslání různých typů zpráv. Komunikátor též obsluhuje čekající zprávy přes `CompletableFuture`.

Komunikátor hry každému připojenému hráči přiřazuje token ve formátu `kód hry + náhodný řetězec`, který slouží k identifikaci hráče po znovupřipojení. Pokud se hráč chce znovu připojit, tak klient pošle kód, který si uložil do `localStorage`, a komunikátor ho podle něj identifikuje a spojí ho s jeho hráčem. Ke každému hráči může být pouze jedno spojení, takže pokud se hráč pokusí připojit s tokenem, který už je připojený, tak se původní odpojí a nahradí ho nový. To řeší situaci, kdy uživatel nechal starou hru otevřenou a zapomněl na to kde.

Komunikátor si hlídá hráče, kteří se odpojili, a pokud se odpojí všichni hráči a nepřipojí se do určitého časového limitu, tak hru smaže, aby se nezdržovala v paměti. Tento časový limit je nastaven na 5 minut, což by mělo být dostatečné pro většinu případů, kdy se hráči odpojí kvůli technickým problémům.

8.2 Interface pro balíčky HerniPlugin a HerniPravidla

Engine je navržený tak, že v základu neobsahuje žádnou hru. Pouze obecné utility a funkce. Z toho důvodu je potřeba, aby hra načítala jednotlivé pluginy. To zajišťují třídy `NacitacPluginu` a `SpravceHernichPravidel`. Aby tyto třídy fungovaly, musí každý plugin implementovat `HerniPlugin`. Ten vrací název hry, popis hry, odkaz na pravidla a hlavně metodu, která vytvoří `HerniPravidla`. `HerniPravidla` jsou hlavní třídou pluginu, jejíž metody hra volá, aby se chovala, jak autor pluginu očekává. Mezi některé její metody, které může autor používat, jsou:

`void pripravBalicek(Balicek<Karta> balicek)` Tato metoda se volá při přípravě hry a měla by naplnit dobírací balíček Kartami.

`public UIPrvek[] getViditelnePrvky()` `getViditelnePrvky()` je metoda, která vrací seznam prvků, které hráči mohou v klientovi vidět. Přestože by měl být server oddělený od logiky klienta, tak je tato metoda potřebná, aby například ve hře, která nepoužívá princip životů, nezabíraly na obrazovce místo ukazatele životů.

`boolean hracChceUkoncitTah(Hrac kdo)` Metoda, která se volá poté, co hráč klikne na tlačítko ukončit tah. Vrací boolean, který určuje, zda hráč tah ukončit může nebo ne. Spousta her hráči

neumožní ukončovat tah, když chce, protože hráč musí například odhodit kartu, proto by tyto hry měly v metodě `return false`. Jiné hry ale nechají hráče udělat kolik akcí chce, a proto by jen například kontrolovaly, jestli nemá hráč v ruce kartu, kterou před koncem tahu musí odhodit.

zacalTah() `zacalTah` je metoda, ve které si může plugin připravit vše, co se odehrává na začátku každého tahu.

Třída obsahuje mnoho dalších metod, které jsou ale principiálně podobné, proto není třeba je zde vysvětlovat. Kompletní seznam možností lze najít v [javadoc dokumentaci](#).

8.3 Karta

Karta je abstraktní třída, jejíž objekt reprezentuje jednu konkrétní kartu ve hře. Každá karta musí mít jméno a identifikátor obrázku (cesta k obrázku bez přípony souboru). Každému objektu karty je automaticky přiřazeno unikátní id, které umožňuje její identifikaci v rámci serveru. Objekty karet jsou klíčovou součástí celého systému karetních her.

Karta sama o sobě nejde zahrát ani vyložit. K tomu poslouží interface `HratelnaKarta` a `VylozitelnaKarta`.

8.3.1 HratelnaKarta

`HratelnaKarta` je interface, který musí implementovat všechny karty, které může hráč zahrát. Obsahuje metodu `boolean odehrat(Hrac kdo)`, která se volá, když se hráč pokusí kartu zahrát. Vrací `boolean`, zda se karta může zahrát (a rovnou již proběhl její efekt), nebo se v současném kontextu zahrát nemůže.

8.3.2 VylozitelnaKarta

`VylozitelnaKarta` je interface, který musí implementovat všechny karty, které může hráč vyložit. Obsahuje metodu `boolean vylozit(Hrac kdo, Hrac kym)`, která se volá, když se hráč pokusí kartu vyložit. Vrací `boolean`, zda se karta může vyložit (a rovnou se k tomu připraví všechny náležitosti), nebo se v současném kontextu vyložit nemůže. Pokud vrátí `true`, tak je na `VylozitelnouKartu` zavolána ještě její metoda `getEfekt()`, která vrací objekt implementující interface `Efekt`. Ve většině případů se jedná o objekt karty, ale autor pluginu se může rozhodnout, že logiku karty a efektu bude držet odděleně, například pokud stejné efekty používá i nějaká postava.

8.4 NacitacPluginu a SpravceHernichPravidel

`SpravceHernichPravidel` je třída, která si udržuje seznam všech objektů `HerniPlugin`, které jsou na serveru dostupné. Obsahuje metody `getJSONvytvoritelneHry()`, která vrací `JSON` objekt obsahující informace o všech dostupných pluginech, které se vyvolají z [Herních pluginů](#) a jejich metod. Dále obsahuje metodu `vytvorHerniPravidla(int id, Hra hra)`, která podle id z `JSONu` zavolá vytvoření pravidel na hře. Tato metoda se využívá vždy, když uživatel vytvoří novou hru, aby do hry napojil správný plugin a jeho pravidla. Třída dále obsahuje metodu `pregeneruj()`, která načte pluginy ze složky `/pluginy/` a uloží je do seznamu. Metoda `pregeneruj` se používá také v `static` bloku, aby se pluginy načetly při startu serveru.

Opravdové načítání pluginů provádí třída `NacitacPluginu`. Ta v metodě `nactiPluginy()` otevře požadovanou složku a pomocí `DirectoryStream` najde všechny soubory s příponou `.jar`. Pro každý soubor zavolá metodu `nactiPluginyZJARu()`, která pomocí `URLClassLoader` načte

třídy z JARu a pomocí `Enumeration` hledá všechny třídy (soubory s koncovkou `.class`). Tyto třídy si načte `ClassLoaderem` a ověří, zda jsou přiřaditelné k `HerniPlugin`, nejsou abstraktní a nejsou rozhraní. Pokud třída splňuje tyto podmínky, tak se vytvoří její instance a přidá do seznamu pluginů. Pokusy o načítání tříd jsou obalené v try bloku, jelikož je v nich mnoho potenciálních chyb, které je potřeba zachytit, aby se načítání pluginů nezastavilo kvůli jednomu špatnému pluginu, například když jeho konstruktor má nějaký parametr, či když neodpovídá stejné verzi *Javy* jako server.

```

1 public static List<HerniPlugin> nactiPluginyZJARu(Path cesta) throws Exception{
2     List<HerniPlugin> pluginy = new ArrayList<>();
3
4     URLClassLoader classLoader = new URLClassLoader(
5         new URL[]{cesta.toUri().toURL()},
6         NacitacPluginu.class.getClassLoader()
7     );
8
9     try (JarFile jar = new JarFile(cesta.toFile())) {
10         Enumeration<JarEntry> entries = jar.entries();
11
12         while (entries.hasMoreElements()) {
13             JarEntry polozkaJARu = entries.nextElement();
14
15             if (polozkaJARu.isDirectory()) {continue;}
16             if (!polozkaJARu.getName().endsWith(".class")) {continue;}
17
18             String nazevTridy = polozkaJARu.getName()
19                 .replace("/", ".")
20                 .replace(".class", "");
21
22             Class<?> c = classLoader.loadClass(nazevTridy);
23
24             if (HerniPlugin.class.isAssignableFrom(c)
25                 && !c.isInterface()
26                 && !Modifier.isAbstract(c.getModifiers())) {
27
28                 HerniPlugin plugin = (HerniPlugin) c.getDeclaredConstructor().
29                     newInstance();
30                 pluginy.add(plugin);
31             }
32         }
33     }
34     return pluginy;
35 }

```

Úryvek 1: Ukázka načítání pluginů z JARu (zjednodušeno)

8.5 Hra a HraImp

`HraImp` je třída implementující interface `Hra`, která obsahuje všechnu obecnou logiku hry, stav hry a přístup k nim pro pluginy. Hru vytváří pomocí tovární metody `vytvor` komunikátor hry, který jí předá id pluginu a sebe, aby mohla komunikovat s klientem. Hra si drží seznam hráčů, dobírací a odhazovací balíček, instanci herních pravidel, správce tahu a podobně.

Hra neřeší ani logiku konkrétní hry, karet, balíčku a akcí hráčů, které řeší Herní pravidla nebo specifické třídy. Hra zařizuje zapnutí hry (`setZahajena(true)`), výhru a konec hry (`skoncil Hrac kdo, vyhral(Hrac kdo)`), přidávání hráčů a gettery pro ostatní obslužné třídy.

Tím, že hra slouží spíše jako lepidlo jednotlivých částí, tak neobsahuje moc funkcí. Její vlastní funkce jsou přidávání hráče, načítání hry, hlídání změny obrázku balíčku a jednoduché utility. Mezi ty patří otáčení balíčků, které jeden balíček otočí a pak prohodí odhazovací a dobírací balíček, nebo sejmutí karty.

Hlídání změny obrázku balíčku se provádí nasazením události – která byla pro tento účel vytvořena – na balíček a nastavením proměnné `vrchniObrazekZadniStrany`, která se pak posílá klientovi, aby věděl, jak balíček zobrazit.

8.6 Správce tahu

SpravceTahu je třída, která se stará o správu tahů ve hře. Přestože se to zdá jako jednoduchý úkol o posouvání ukazatele, tak je třída poměrně složitá. Musí podporovat změnu směru, vynechání tahu, dočasné či trvalé přidání hráče, násobič tahu a podobně.

Pro svoji vnitřní logiku používá třídu **Tah**, která reprezentuje jeden tah ve hře. Tah se znovu používá, to znamená, že jeden objekt nereprezentuje konkrétní tah, ale všechny tahy jednoho hráče. Tah pouze obsahuje veřejné proměnné, které ukládají hráče, jehož je to tah, zda je tento tah jednorázový (tzn. po jeho odehrání se vyřadí z fronty, na rozdíl od normálního tahu, který se opakuje každé kolo) a zda je tah dočasně zrušený (mohlo by se používat například, když má hráč za trest dvě kola nehrát, ale není žádoucí, aby se vyřadil z pořadí).

Třída **SpravceTahuImp** obsahuje frontu (**ArrayDeque**) s objekty tahu.

Pro změnu směru se používá metoda **zmenaSmery()**, která neotáčí celou frontu, ale pouze změní příznak směru, podle kterého se pak rozhoduje, zda se z fronty bere zespoda, nebo zezhora.

Protože je pro některé funkce žádoucí mít k dispozici seznam všech hráčů, kteří nejsou vyřazeni, ve správném pořadí, tak existuje metoda **getHrajiciHraci()**. Původně měla fungovat tak, že by správce tahu držel jak frontu tahů, tak informace o hráčích, to by ale bylo velmi náchylné k chybám, protože by bylo potřeba synchronizovat hned dvě struktury bez chyby. Proto byla zvolena varianta generování seznamu z fronty tahů. Jelikož je tato metoda poměrně náročná, tak se její výsledek ukládá do proměnné sloužící jako cache a aktualizuje se pouze při změně fronty tahů, tudíž jejím zneaktuálněním. Díky tomu pluginy mohou metodu volat klidně vícekrát za sebou bez většího výkonnostního problému.

```
52  @Override
53  public List<Hrac> getHrajiciHraci() {
54      if (!poradiAktualni) {
55          hrajiciHraciCache = new ArrayList<>();
56          for (Tah t : frontaTahu) {
57              if (!t.docasneZruseny && !t.jednorazovy) {
58                  hrajiciHraciCache.add((HracImp) t.hrac);
59              }
60          }
61          if (zmenenSmery) {
62              Collections.reverse(hrajiciHraciCache);
63          }
64          poradiAktualni = true;
65      }
66      return List.copyOf(hrajiciHraciCache);
67  }
```

Úryvek 2: Ukázka získání hrajících hráčů ve správci tahu

Dalším častým problémem je vyřazení hráče a jeho následné vrácení. To zajišťují metody **vyraditHrace(Hrac koho)** a **vratitHrace(Hrac koho)**, které v řadě tahů najdou ten správný a přenastaví mu příznak dočasně zrušený. Je třeba rozlišovat vrácení hráče a jeho přidání, jelikož přidání hráče musí vyrobit nový **Tah** a vybrat mu nové místo ve frontě. Přidání hráče zajišťuje metoda **pridatHrace(Hrac koho)**

```
194  public void vratitHrace(Hrac koho){
195      for (Tah tah : frontaTahu) {
196          if (tah.hrac.equals(koho)) {
197              tah.docasneZruseny = false;
198          }
199      }
200      poradiAktualni = false;
201  }
202  public void pridatHrace(Hrac koho) {
203      frontaTahu.addLast(new Tah(koho, false));
204      poradiAktualni = false;
205  }
```

Úryvek 3: Ukázka vrácení hráče a přidání hráče ve správci tahu

Samotná metoda `dalsiHrac()` musí zkontrolovat všechny příznaky a podle nich rozhodnout, který hráč je na tahu. Metoda také řeší všechny problémové stavy, například když nemá kdo hrát.

8.7 Hrac a HracImp

`HracImp` je třída implementující interface `Hrac`, která reprezentuje jednoho hráče ve hře. Obsahuje informace o hráči, jako je jeho jméno, role, postava, počet životů, karty v ruce, karty na stole a podobně. Dále obsahuje metody pro manipulaci s těmito daty.

Tato třída obsahuje nejvíce metod pro pluginy. Mezi ty patří například `lizniKontrolovane`, `pridejEfekt` a `jeZivy`.

8.7.1 boolean odeberZivot() a boolean pridejZivot()

Odeber život je metoda, kterou by měl plugin volat, když hráč přijde o život. Metoda zkontroluje, zda ještě před odebráním života nemá zavolat nějaký efekt a potom zkontroluje, jestli hráči nedošly životy. Pokud hráči došly životy, tak se nepůjde do mínusu v počtu životů, ale vrátí se `false` a zavolá se `HerniPravidla.dosliZivoty(Hrac kdo)`, aby plugin mohl zareagovat a například zařídit smrt hráče. Pokud hráč ještě životy má, tak se jeden odebere, zavolají se efekty a metoda vrátí `true`.

Přidej život je podobná metoda s opačným účinkem. Místo smrti kontroluje, zda se nepřesáhne dané maximum životů, které hráč může mít.

8.7.2 odehranaKarta(), vylozitKartu() a spalitKartu()

Tyto metody volá komunikátor hry, když se hráč rozhodne zahrát, vyložit nebo spálit kartu. Nejsou určené pro plugin, protože obsahují všechny kontroly pro vstup od uživatele a jako parametr berou *String* id karty a ne samotný objekt, což je v rámci serveru neefektivní.

Metoda `odehranaKarta(String id)` si převede id do *intu*, ověří, že je hráč na tahu a pokusí se kartu najít mezi kartami v ruce porovnáváním id. Poté zkontroluje, zda je karta hratelná (instance `HratelnaKarta`), přetypuje ji a zavolá její metodu `odehraj(Hrac kdo)`. Pokud se karta zahrála, přesune ji na odhazovací balíček a upozorní komunikátor hry.

U této metody bylo potřeba být obzvlášť opatrný ohledně validace, protože přijímá číslo z klienta a musí zkontrolovat zda se jedná o číslo, zda taková karta existuje a podobně.

Metody `vylozitKartu(String id, String idHrace)` a `spalitKartu(String id)` fungují podobným způsobem, ale s příslušnou logikou pro vyložení a spálení karet.

8.7.3 vzdalenostPod(int max, boolean zpetne)

Tato metoda má několik obdobi, které například vrací pouze vzdálenost k jednomu hráči, nebo vzdálenost bez efektů, ale tato verze vrací všechny hráče, kteří jsou v dané vzdálenosti, nebo blíž. Vzdálenost se počítá jako počet hráčů mezi dvěma hráči ve směru hodinových ručiček. Pokud je `zpetne true`, tak se počítá vzdálenost proti směru hodinových ručiček.

```
566     @Override
567     public List<Hrac> vzdalenostPod(int max, boolean iZpetne) {
568         List<Hrac> hraci = hra.getSpravceTahu().getHrajiciHraci();
569         List<Hrac> vysledniHraci = new ArrayList<>();
570
571         int velikost = hraci.size();
572         int i1 = hraci.indexOf(this);
573     }
```



```

574         if (i1 == -1) {
575             throw new IllegalArgumentException("čHrá nebyl nalezen v seznamu");
576         }
577
578         int mujDosah = this.getCelkovyBonusDosahu();
579
580         for (int i = 0; i < hraci.size(); i++) {
581             // Nezahrne sebe
582             if (i == i1) {
583                 continue;
584             }
585
586             Hrac cil = hraci.get(i);
587             int rozdil = Math.abs(i1 - i);
588             int fyzickaVzdalenost;
589
590             if (iZpetne) {
591                 // Bere menci vzdalenost (Budto zleva, nebo zprava)
592                 int zpetnaVzdalenost = velikost - rozdil;
593                 fyzickaVzdalenost = Math.min(rozdil, zpetnaVzdalenost);
594             } else {
595                 fyzickaVzdalenost = rozdil;
596             }
597
598             // Spocitani efektu
599             int odstupCile = cil.getCelkovyBonusOdstupu();
600             int efektivniVzdalenost = fyzickaVzdalenost + odstupCile - mujDosah;
601
602             // Osentreni maximální vzdalenosti
603             efektivniVzdalenost = Math.max(1, efektivniVzdalenost);
604
605             // Pridani do seznamu, pokud je na dostrel
606             if (efektivniVzdalenost <= max) {
607                 vysledniHraci.add(cil);
608             }
609         }
610
611         return vysledniHraci;
612     }

```

Úryvek 4: Počítání vzdálenosti hráčů

9 Implementace pluginů

K enginu bylo vyrobeno několik pluginů – Bang!, Prší a UNO. Pluginy jsou v repozitáři ve složce /pluginy/.

9.1 Bang!

Tento plugin je pro projekt klíčový, jelikož je engine přizpůsobený hlavně jemu. V původní fázi byl dokonce do enginu hardcodován. Teď už je většina jeho prvků přesunuta do jeho herních pravidel.

V Bangu bylo potřeba zajistit, aby měl správně nakonfigurovaná pravidla. Dále bylo potřeba implementovat všechny karty, efekty a postavy.

Mezi nejsložitější část pluginu patřila implementace karet *Bang!* a *Kulomet*. Přestože jejich efekt vypadá jednoduše, tak je potřeba zkontrolovat, jestli cíl útoku nemá *Barel*, popřípadě ho nechat zahrát *Vedle*. Protože jsou tyto karty velice podobné, tak jejich funkčnost zajišťují metody z *PravidlaBang*, které se volají z karet. Omezení počtu zahrání karty Bang za jeden tah se původně řešilo okamžitým ukončením tahu po zahrání karty. To ale komplikovalo limit počtu karet na konci tahu a mohlo pro hráče působit neočekávaně, proto teď karta Bang pouze nastaví pravidlům příznak, že už byla zahrána a při příštím pokusu o zahrání se tento příznak zkontroluje.

V pluginu Bang! bylo potřeba vytvořit velké množství karet a postav. Pro to šel použit engine naplno a díky tomu, že efekty mohou být samostatnými třídami, tak postavy a vyložitelné karty, které sdílejí společnou logiku, mohou mít společnou třídu efektu, což zjednodušilo implementaci a pomohlo dodržovat *DRY* princip.

Tento plugin není samostatný a součástí balíčku má taky variantu, ve které platí trochu jiná pravidla. Tyto verze budou moci být sloučeny do jedné, až bude vytvořena možnost konfigurovat pravidla před zahájením hry. Už teď to ale nevytváří žádné duplicity, protože třída pravidel a karty jsou stále stejná, liší se pouze argumenty konstruktoru.

9.2 Prší

Prší je hra, která byla kompletně přidána jako první a používala se pro testování.

Implementace karet byla jednoduchá. Existuje základní karta **PrsiKarta**, která má svoji barvu (**PrsiBarva**) a hodnotu (**PrsiHodnota**). Karta je implementace interface **HratelnaKarta**. Metoda volaná při odebrání neprovádí žádný speciální efekt, pouze kontroluje předchozí kartu v odhazovacím balíčku, zda se shoduje hodnotou, barvou, či zda vůbec není instancí Prší karty (To by nastat nemělo, ale kdyby nějaký balíček kombinoval karty, tak se díky tomu nic nerozbije).

Tuto základní kartu implementuje **PrsiSedmicka**. Ta se liší v tom, že následující hráč si musí lízat, či sedmičku přebít. Naprogramovat tuto kartu samostatně nejde, protože mění globální stav hry. Z toho důvodu bylo třeba do herních pravidel přidat pro sedmičku metodu **zahranaSedmicka**. **PravidlaPrsi** si drží proměnnou **pocetKaretNaLiznuti**, která se postupným přebíjením zvyšuje a v okamžiku, kdy si někdo lízne, se mu dolízne karet více.

Dalším potomkem Prší karty je **PrsiSvrsek**. Ten se liší v tom, že lze vyložit nad libovolnou kartu, tudíž jeho metoda **odehraj()** vrátí vždy **true**. Než to ale udělá, tak u hráče, co ji zahrál, vyvolá dialog na výběr barvy. Do doby než hráč vybere barvu, tak jeho getter pro ostatní karty vrací **null**. Po zvolení barvy se svrsek změní ze svojí původní barvy na barvu, kterou hráč vybral. Také se ostatním pošle upozornění, jaká barva byla vybraná.

Poslední speciální kartou je **PrsiEso**. Jelikož se v programované verzi prší eso nepřebíjejí, tak eso po zkontrolování podmínek zahrání pouze zavolá **hra.getSpravceTahu().eso()**, jelikož eso je funkce v enginu zabudovaná.

9.3 Uno

Poslední implementovaná hra je UNO, které je pouze starší verzí Prší, bez další složitě naprogramované mechaniky. V současné verzi UNO neobsahuje změnu směru, „+2“ a „+4“, ale funkce těchto karet engine podporuje, proto by v případě zájmu nebyl problém je přidat.

Na tomto pluginu je nejzajímavější generování obrázků jeho karet pomocí Pythonu.

9.4 Budoucí pluginy

V následující době bude vytvořeno Kvarteto, které bude využívat dialogy typu texty, Výbušná kofátka, která využijí možnosti složitější logiky karet a rozšíření Bangu, které využije možnost mít více pluginů v jednom balíčku.

10 Protokol

Pro komunikaci mezi klientem a serverem se používá websocketový protokol. Již od začátku se posílají textové řetězce ve formátech:

```
1 <typZpravy>:<data>
2 <typZpravy>
3 <typZpravy>:<údaj1>,<údaj2>,...,<údajN>
4 <typZpravy>:<json>
```

Úryvek 5: Formáty komunikačních zpráv systému

Tento formát sice nepatří mezi nejefektivnější a řešením jednotlivých bitů by šel velmi zkrátit, ale pro množství poslaných zpráv a jejich velikost by to bylo zanedbatelné. Takhle je formát jednoduchý a přehledný, takže i při testování lze psát zprávy ručně a není potřeba žádný nástroj pro serializaci a deserializaci.

Hlavní problém protokolu je, že nepojmenovává jednotlivé typy zpráv vždy stejným způsobem, což může být matoucí. Tento problém vznikl tím, že se protokol vyvíjel postupně a nebyl od začátku přesně definovaný.

Průběh komunikace začíná první fází tak, že server uvítá klienta, potom klient požádá o seznam dostupných her a server mu ho dodá. Pak si hráč vybere nějakou akci připojení (vracení se, vytvoření hry, připojení se ke hře) a řekne své jméno.

```
1 -> welcome
2 <- infoHer
3 -> infoHer:{ "verze": "0.0.7", "hry": [{ "id": 0, "jmeno": "UNO!", "popis": "Slavná základní
   karetní hra.", "url": "" }, { "id": 1, "jmeno": "šPri", "popis": "Česká čtradiní hra, ve které
   se žsnaíte zbavit švech karet, které ůžmete zahrát podle stejné barvy nebo hodnoty.",
   "url": "" }, { "id": 2, "jmeno": "Bang!", "popis": "Populární hra s cílem zabít ostatní
   čhráe.", "url": "https://albi.cz/data/files/products/24327/1603711687-bang-pravidla-
   zakladni-hry.pdf" } ] }
4 <- pripojeniKeHre:123456
5 <- noveJmeno:Honza
6 -> pripojenKeHre
```

Úryvek 6: Příklad průběhu 1. fáze komunikace

V druhé fázi už je hráč připojen ke hře, která ale není zahájena. V tuto chvíli se načítají různé informace o hře a hráčích, kteří se mohou i postupně připojovat. Klientovi bude přiřazen znovupřipojovací token, který mu umožní se znovu připojit ke hře, pokud se odpojí, a jeho id hráče, aby věděl, jaké informace se týkají přímo jeho.

```
7 -> token:123456_08gs7oLEqthBeaGkelKVAnl-f_CN5YoctfE7rdOnek
8 -> vyberPostavu: [{ "jmeno": "TESTOVACÍ", "obrazek": "TESTOVACI", "popis": "pouze pro test",
   "zivoty": "4" }, { "jmeno": "TESTOVACÍ", "obrazek": "TESTOVACI", "popis": "pouze pro test",
   "zivoty": "4" } ]
9 -> novyHrac: { "id": 4, "jmeno": "nepojmenovaný čhrá", "zivoty": 0, "pocetKaret": 0,
   "maximumZivotu": 0 }
10 -> noveIdHrace: 4
11 -> povoleneUI: [ "ODHAZOVACI_BALICEK", "DOBIRACI_BALICEK" ]
12 -> noveJmeno: 4, honza
13 -> novyHrac: { "id": 5, "jmeno": "nepojmenovaný čhrá", "zivoty": 0, "pocetKaret": 0,
   "maximumZivotu": 0 }
14 -> noveJmeno: 5, Adam
15 <- setPostava: 1
```

Úryvek 7: Příklad průběhu 2. fáze komunikace

Třetí fáze už je samotná hra. Klient posílá akce hráče, zatímco server mu vrací svoje vypočítané reakce, aktualizace stavu hry a akce jiných hráčů.

```
16 <- zahajeniHry
17 -> novaKarta: { "jmeno": "red9", "obrazek": "uno/red9", "id": 109 }
18 -> novaKarta: { "jmeno": "yellow6", "obrazek": "uno/yellow6", "id": 136 }
19 -> role: BANDITA
20 -> novyPocetKaret: 5, 8
21 -> hraZacala
```

```

22 -> zahZacal:5
23 [druhý hráč odehraje]
24 -> odehrat:5|{"jmeno":"yellow9","obrazek":"uno/yellow9","id":139}
25 -> novyPocetKaret:5,7
26 -> tvujTahZacal
27 [tento hráč si lízne]
28 <- linuti
29 -> novaKarta:{"jmeno":"eso","obrazek":"uno/eso","id":148}
30 -> tahZacal:5

```

Úryvek 8: Příklad průběhu 3. fáze komunikace

11 Implementace klienta

11.1 O aplikaci

Jak již bylo zmíněno, klient je napsán pro framework *React* pro web. Obsahuje hlavní soubor `main.tsx`, který obsahuje různé providery a celoaplikační komponenty. Hlavní komponentou celé aplikace je `App`, která přepíná mezi stránkami a řeší lazyloading. Jednotlivé stránky se nacházejí v `/src/pages/`.

Hlavní kontext, který obsahuje jak funkce pro celou aplikaci a všechna data, je `GameKontext`. Ten obsahuje jednotlivé funkce pro posílání dat na server, úpravu dat, akce hráče a podobně. Jeho provider se jmenuje `GameProvider`. V něm nejsou implementovány tyto funkce, ty jsou spolu s logikou příchozích zpráv přemístěny do `gameActions`. Provider ovšem obsahuje efekt, pomocí něhož se připojí k websocket serveru.

Jednotlivé komponenty, které něco vizuálně ukazují, se nacházejí ve složce `/src/components/`. Některé z těchto komponent jsou přímo napojené na herní kontext a zobrazují aktuální data, zatímco jiné jsou obecnější a zobrazovaná data si vezmou z parametru.

11.2 Hlavní objekt `gamestate`

`GameState` je objekt držený v `GameKontextu`, který obsahuje všechny informace o probíhající hře. Objekt též obsahuje informaci, zda je hráč připojen ke hře (`inGame`), zda je hra spuštěna (`gameStarted`) a seznam pluginů, které jsou na serveru nainstalované, včetně jejich metadat (`gameTypesAvailable`).

Tento objekt používá celá aplikace, aby mohla přistupovat k aktuálním datům. Jednotlivé komponenty a akce hráče ovšem neupravují objekt napřímo, ale volají funkce akcí, které jsou součástí kontextu. Tyto funkce často komunikují se serverem, nebo upravují kontext. Spolu s metodou na přijímání zpráv od serveru se nacházejí v souboru `gameActions.ts`.

11.3 `gameActions.ts`

Game actions je soubor exportující metody jednotlivých akcí, které hráč může provést a reakce serveru na ně nebo akcí, které nadiktoval server od ostatních hráčů.

Hlavní metoda je `handleGameMessage(event:MessageEvent,...)`, která se volá, když je přijata zpráva od serveru. Funkce si nejprve rozdělí zprávu na `type` a `payload` (viz protokol) a v jednom swich bloku zpracuje všechny typy. Ty často zahrnují volání `setGameState()`, kterým se aktualizuje stav hry. Jednotlivé bloky si mohou parsovat `payload` podle sebe, aby dodržovaly formát v protokolu.

Další důležitou částí souboru jsou exporty metod pro hráčské akce. Tyto funkce vykomunikují všechno se serverem, aby to nemusely dělat jednotlivé komponenty, a všechno, co se týká protokolu,

se nacházelo na jednom místě. Zde byla na rozdíl od serveru výhoda, že se s touto logikou počítalo od začátku a nenastaly žádné zmatky.

11.4 Připojovací stránka LoginPage

Tato stránka by pro uživatele měla být první stránkou, kterou uvidí. Obsahuje nějaké základní informace o hře a formuláře pro připojení se. Tyto formuláře existují tři – první z nich slouží pro připojování se ke hře a zobrazí se jen pokud má hráč v `localStorage` uložený token hry, který mu je přidělen v průběhu připojování se. Druhý slouží k připojení se pomocí kódu hry a jména a třetí slouží k vytvoření hry. U vytvoření hry je výběrová nabídka dostupných pluginů, která se musí nejdříve dostat od serveru, aby se mohly zobrazovat názvy her, popisy a pravidla.

11.5 Herní stránka GamePage

Herní stránka je stránka, která se zobrazuje, když je hráč připojen ke hře a hra je spuštěna. Obsahuje všechny herní prvky, které jsou zobrazené hráči. Kromě prvků obsahuje utility drag & drop kontext `DndContext` a `<GlobalNotifications />`.

Stránka se dělí do tří částí pod sebou – panel hráčů **Players** s informacemi ke každému hráči, centrální panel **CentralPanel** s kartami na stole a dalšími prvky společné pro všechny hráče a panel s prvky související pouze s konkrétním aktivním hráčem **MyThings**.

V této stránce se nacházejí všechny komponenty, které se přímo týkají hry.

11.6 Drag & drop

Hra používá drag & drop logiku pro přesun karet, díky které může hráč jedním pohybem určit, jak co a kam chce předělat mnohem přirozeněji než několika kliknutími. Pro tento účel se používá knihovna *react-dnd*. Původně bylo učiněno několik pokusů o vlastní implementaci, ale nakonec se ukázalo, že pro tento projekt je lepší použít hotové řešení, které je dobře zdokumentované, podporované a obsahuje řešení všech hraničních případů i typů uživatelského vstupu.

11.7 Notifikace

Hra používá více systémů notifikací. První z nich jsou toast notifikace v pravém horním rohu pro informování hráče. Používají se, když se stane nějaká událost, která nemusí být jasně patrná z drobné změny UI, která proběhla. Druhým typem jsou notifikace, které se zobrazují přímo uprostřed. Cílem těchto notifikací je upozornit na sebe, i když hráč například nedává pozor. Tyto notifikace se používají například pro upozornění na začátek tahu.

Pro toasty se používá knihovna *react-hot-toast*, která poskytuje jednoduché API pro zobrazení toastu. Knihovna byla vybrána i přesto, že by bylo možné tuto funkčnost vyrobit bez ní, ale nebyl důvod „znovu vynalézat kolo“. Toast jde po importu vyvolat pomocí funkce `toast()`, která má několik variant pro různé typy notifikací, například `toast.error()` pro chybové notifikace.

Pro notifikace uprostřed se používá vlastní komponenta **CentralNotification**, která zobrazuje zprávu jako jednoduchý text, který se uprostřed obrazovky začne zvětšovat a zprůhledňovat. Komponenta využívá portál, kterým do `<body>` stránky vkládá absolutně pozicovaný element, do kterého vykresluje samotnou notifikaci. Po importu komponenty lze notifikaci vyvolat metodou `notify()`.

Pokud je třeba hráče informovat a on musí přečtení potvrdit, tak se použije [dialog](#).

11.8 Dialog

Dialog je komponenta se svým kontextem, která poskytuje API pro informování hráče a získávání vstupu od něj. Skládá se z `DialogProvideru`, který vytváří kontext a poskytuje metody `openDialog` a `closeDialog`. `DialogProvider` vrací kontext `DialogContext`, který exportuje tyto metody a potřebné typy. Samotná komponenta `Dialog` používá kontext dialogu a v případě otevření vytváří portál, ve kterém vytváří obal dialogu, ztmavené pozadí a samotný obsah dialogu. Ten se určí podle typu dialogu. Existují typy `SELECT_CARD`, `SELECT_PLAYER`, `CONFIRM`, `TEXT`, `SELECT_ACTION` a `INFO`. Každý z těchto typů má své specifické vlastnosti, například `SELECT_CARD` obsahuje seznam karet, ze kterých může hráč vybírat, zatímco `CONFIRM` obsahuje pouze zprávu a tlačítka pro potvrzení nebo zamítnutí.

Dialog má různé možnosti, například zda se může zavřít bez akce, kolik položek může maximálně i minimálně vybrat a podobně.

11.9 Tmavý režim

Celý web je vyroben s podporou tmavého režimu. Pro tmavý režim se nepoužívá `prefers-color-scheme`, aby uživatelé mohli používat přepínač v aplikaci. Stav tmavého režimu ovládá přidáváním třídy `darkMode` na `<body>` element. Nepoužívá se kontext, aby se nemusel překreslovat framework a všechny komponenty, ale pouze `CSS` interně v prohlížeči. V `.module.css` souborech se pak používá `:global(.darkMode)` pro funkčnost i po kompilaci, která by jinak změnila názvy tříd.

Přepínač je komponenta `DarkModeSwitch`, která se vykresluje jako měsíček či sluníčko v pravém horním rohu. Tento přepínač se neukazuje na normální hrací stránce, kde by byl zablokovaný, ale režim lze přepínat například při otevřeném dialogu.

11.10 Server info a game info

Nepřímou součástí klienta jsou i `HTML` stránky `serverInfo.html` a `gameInfo.html`, které se nachází v `/public/` adresáři. Server info slouží pro zobrazení informací o serveru, které jsou užitečné pro správu serveru, například pro zjištění aktuálního počtu připojených hráčů, seznamu her a podobně. Stránka získává data pomocí websocketového připojení k serveru a zobrazuje je v jednoduchém formátu, který poskytuje server.

Game info naopak slouží pro zobrazení informací o konkrétní hře, které jsou užitečné pro testování a ladění pluginů. Tato stránka zobrazuje informace o stavu hry, hráčích, kartách a podobně. Stejně jako server info získává data pomocí websocketového připojení. Na rozdíl od server info server neposílá informace jako `HTML` stránku, ale jako `JSON`, který se potom zpracuje a zobrazí v přehledném formátu. Tato stránka je užitečná pro testování a ladění pluginů, pokud například není schoda mezi reálným stavem hry a stavem hry, který si vykládá klient.

Tyto části vyžadují ověření administrátorským heslem od serveru, protože zobrazují citlivé informace o hráčích a hře, které by neměly být přístupné pro běžné hráče.

11.11 Původní klient

V zářijové fázi projektu se místo současného klienta v `Reactu` používal klient ve *vanilla JavaScriptu*. I přesto, že již není vyvíjen, tak některé základní funkce stále podporuje, a proto se v repozitáři stále nachází ve složce `klient`.

12 Utility pro vývoj

Kromě hlavních částí projektu bylo během vývoje vyrobeno několik užitečných python programů, které pomáhaly s různými úkoly. Například pro generování obrázků karet byl pomocí umělé inteligence vytvořen python skript, který pomocí knihovny *PIL* generoval obrázky karet na základě zadaných parametrů. Tento skript umožnil rychle vytvořit velké množství karet pro různé pluginy bez nutnosti ručního designování každé karty. Další užitečný skript stahoval karty z prší do konkrétní složky, což urychlilo proces získávání potřebných obrázků pro tento plugin.

13 Build, hosting a nasazení

Serverová část projektu se kompiluje pomocí *Mavenu*. Build není potřeba zařizovat přímo. Stačí použít připravený skript `build.sh` v kořenovém adresáři. Tento skript vytvoří složku `build` se zkompilevaným serverem, složkou s pluginy a skriptem `start.sh` pro spuštění serveru. Pro build klienta stačí v její složce spustit `npm run build`, což vytvoří `dist` s připravenými soubory pro server.

Před buildem klienta a spuštěním serveru je potřeba správně nakonfigurovat oba `.env` soubory na stejný port a doménu.

Jak již bylo zmíněno, tak je hra hostovaná na studentském serveru Gymnázia Arabská *avava*. Tento server poskytuje *Caddy* server, který se při požadavku chová nejprve jako statický server pro soubory, ale jakmile nějaký soubor neexistuje, tak předá požadavek na backendový server tím, že zavolá na konkrétní port. Na tomto portu běží server projektu s websocketem. Aby se nevypnul při odhlášení se z SSH, tak se používá nástroj *tmux*, který umožňuje spouštět procesy na pozadí a připojovat se k nim později. Na statické části serveru je uložen i předem postavený klient, který se načítá hráčům, když navštíví webovou stránku. Pro připojení k websocketu se používá `/ws` endpoint.

14 Testování

Během vývoje byla hra nesčetněkrát testována jejím autorem. Kromě toho se mnohokrát testovala i s jeho kamarády a rodinou, kteří hru zkoušeli a upozorňovali na nedostatky. Hra se testovala jak na lokálním serveru, tak i na veřejném serveru, aby se otestovalo chování v různých sítích a s různými počty hráčů.

Díky tomu, že se do enginu přidaly rychle jednodušší hry, tak bylo testování zábavné, a proto se mohlo testovat často a s různými lidmi, což pomohlo odhalit mnoho nedostatků a zlepšit hru.

Pro testování existuje i několik nástrojů, například pro zobrazení boxů kolem všech prvků DOM stačí přidat body třídu `testovani`, nebo pro zobrazení celé websocketové komunikace stačí otevřít konzoli prohlížeče a podívat se na zelené zprávy. Také existují pomůcky `serverInfo.html` a `gameInfo.html`.

15 Řešené problémy

Během vývoje se vyskytlo více problémů, které se musely řešit, v této kapitole jsou popsány ty zajímavější z nich.

Největším problémem byla migrace SDK, které původně neexistovalo. Jakmile se ukázala jako nutnost, tak bylo potřeba vytvořit nové rozhraní, rozhodnout, co je pro pluginy důležité, přetypovat některé metody v rámci serveru, přepsat v té době rozpracované pluginy, aby přešly na nový, lepší

systém a provést refactoring názvů tříd serveru, aby se neshodovaly s názvy rozhraní v SDK. Názvy se nakonec vyřešily přidáním přípony `Impl` pro implementace a ponecháním názvů pro rozhraní.

Dalším problémem bylo vytvoření asynchronní logiky dialogů, která umožní hráčům vybírat karty, hráče a podobně. Tento problém se řešil pomocí `CompletableFuture`, který umožňuje vytvořit asynchronní logiku, která lze jednoduše použít v rámci enginu a pluginů.

Plugin Bang! pro svoji funkčnost potřebuje často znát vzdálenost mezi dvěma hráči. Je to sice logika specifická pro Bang!, ale protože se jedná o logiku, která se může hodit i jiným pluginům, tak byly do třídy `Hrac` přidány metody na počítání těchto vzdáleností. Problém, který nastal byl, jak vyřešit karty *Mustang* a *Hledí*, které tyto vzdálenosti mění. Nakonec byl problém vyřešen tak, že efekty dostaly možnost posunutí vzdálenosti o určitý počet míst. Tyto metody musí být dvě podle směru, protože *Hledí* posouvá pouze vzdálenost, na kterou vidí ostatní hráče, zatímco *Mustang* mění pouze tu, na kterou je viděn ostatními. Při počítání vzdáleností se potom tyto posuny sečtou ze všech efektů a zohlední. Při počítání vzdáleností bylo také potřeba vzít ohled na to, že `List` hráčů není uložen jako kruh, ale jako řada. Tato problematika byla vyřešena pomocí funkcí `min` a `max`.

Při programování některých funkcí bylo využito funkcionálního programování pomocí `stream()` metody, která umožňuje pracovat s kolekcemi dat pomocí funkcionálních operací jako jsou `map`, `filter`, `reduce` a podobně. Toto řešení pomohlo zjednodušit kód a udělat ho přehlednější, protože se místo psaní složitých smyček a podmínek použily jednoduché funkcionální operace. Problémem u tohoto řešení bylo, že se někdy chovalo neočekávaně a hůře se opravovalo. Funkcionální programování bylo také využito u různých metod, do kterých se hodilo mít možnost předat další funkci jako parametr. Nejprve bylo složité se v tom vyznat, protože narozdíl od *JavaScriptu*, má Java více druhů funkcí podle počtu parametrů a toho, co vrací. Nakonec se ale podařilo vytvořit systém, který umožňuje předávat funkce jako parametry a využívat tak výhod funkcionálního programování.

Projekt obsahuje možnost vrácení se do hry po nenadálém odpojení při výpadku sítě, nebo například zavření okna. Pro tento účel musel být zaveden znovupřipojovací token, který je při připojení se ke hře vygenerován a poslán klientovi. Tento token je pak uložen v `localStorage` a při načítání stránky se zkontroluje, jestli tam není. Pokud tam je, tak se uživateli nabídne možnost se vrátit do hry, což po potvrzení pošle serveru token a id hráče, aby se mohl znovu připojit ke hře. Kvůli této funkcionalitě nastal problém se synchronizací stavu, kdy se při připojení musí načíst úplně všechna data. Nejprve mnoho dat chybělo, ale postupně se přidávaly další a další, až se nakonec podařilo načítat všechna data, která jsou potřeba pro správné zobrazení stavu hry.

U třídy `HraImpl` byl problém v tom, že při vytváření musela provést některé akce, které ale v konstruktoru být nemohly, protože se ještě nedokončila inicializace objektu. Tento problém se řešil pomocí statické tovární metody `vytvor()`, která hru vytvoří a dané akce poté provede.

Také se kvůli několika příkazům, které jsou ověřeny administrátorským heslem, musela vytvořit ochrana proti brute-force útokům. Ochrana se řeší čtyřsekundovým globálním zámkem, který je aktivovaný během zadávání hesla. Pokud je zámek aktivní, tak se žádný další pokus o zadání hesla neprovede, dokud zámek nevyprší. Toto řešení sice neodolá *ddos* útokům, to ale nevadí, protože ověření hesla není kritickou funkcí, u které by vadila její nedostupnost.

Dalším velkým problémem bylo vytvoření *CSS* pro klienta, který by byl přehledný, udržitelný a dobře strukturovaný. Tento problém se řešil pomocí *CSS Modules*, které umožňují psát *CSS* v jednotlivých modulech, které jsou pak importovány do komponent. Díky tomu se *CSS* stalo přehlednější a udržitelnější, protože každý modul obsahuje pouze *CSS* pro jednu komponentu. Dále existuje globální modul, který obsahuje užitečné třídy – například tlačítka.

Pro klienta se též musel řešit problém, jak fungují pokročilejší funkce Reactu, se kterými se autor nikdy předtím nesetkal.

16 Budoucí rozšiřování

Projekt je navržený tak, aby se dal snadno rozšiřovat. Díky pluginům je možné přidat další hry, které budou fungovat na stejném enginu. V plánu je přidat některá rozšíření pro *Bang!*, která jsou již předpřipravená. Z nových her jsou promyšlená Výbušná koťátka a Kvarteto, která využijí některé funkce enginu, které se ještě nevyužívají. Dále by bylo možné přidat i další funkce pro klienta, například možnost přizpůsobení vzhledu, přidání zvuků a podobně. Pro klienta je v plánu přidat *peer-to-peer* voice chat, který pomůže udělat hru akčnější a zábavnější. Tento chat bude fungovat pomocí *WebRTC*. To umožňuje přímou komunikaci mezi prohlížeči bez potřeby serveru pro přenos dat. Server bude sloužit jako zprostředkovatel přenosu připojovacích tokenů z knihovny *PeerJS*.

Pro server se v delší budoucnosti plánuje vylepšit SDK, aby podporovalo více herních mechanik. Dále se přidá možnost nastavení před hrou, která umožní pluginům vyrobit objekt variant pro nastavení volitelných parametrů hry. Tento objekt získá klient, který hru založil, a ten ho zobrazí jako formulář pro nastavení hry.

V současné verzi je tvorba nových pluginů omezena na jazyk *Java*. V budoucím vývoji je plánována implementace polyglotní architektury, která umožní psát pluginy v jazycích *JavaScript* a *TypeScript*. Tato změna umožní vytváření pluginů i v populárnějších jazycích. Propojení bude uděláno pomocí *GraalVM Polyglot API*, které umožňuje spouštění skriptů s vysokým výkonem. Kritickým požadavkem pro tuto funkcionalitu je zajištění bezpečnosti serveru. *JavaScriptové* pluginy proto budou spouštěny v přísně izolovaném prostředí (*sandboxu*), které zamezí přístupu k systémovým prostředkům a povolí interakci pouze s explicitně exportovaným rozhraním enginu (*API whitelist*). Díky tomu bude teoreticky možné povolit přidávání pluginů přímo přes web bez nutnosti schválení a nahrání administrátorem.

Poslední zvažovanou možností je vytvořit knihovnu, která bude obstarávat komunikaci z pohledu klienta a tím umožnit vytváření herních botů.

17 Závěr

Cílem této ročníkové práce bylo navrhnout, naprogramovat a do reálného provozu nasadit síťový herní engine s architekturou klient-server, který umožní hraní tahových karetních her v prohlížeči. Všechny předem stanovené cíle byly úspěšně splněny. Vznikl plně funkční a stabilní systém schopný obsluhovat více hráčů a herních místností současně.

Během vývoje bylo nutné překonat řadu technických a architektonických výzev. Na straně serveru to byl hlavně návrh vlastního komunikačního protokolu nad technologií *WebSocket*, správa asynchronních tahů hráčů a implementace dynamického načítání tříd (*ClassLoader*), které umožňuje naprosté oddělení logiky enginu od konkrétních herních pravidel a správné rozdělení úkonů mezi plugin a server. Na straně klienta byla nejsložitější správa dat a vytvoření celého komplexního systému v *Reactu* vytvořeného pro udržitelnost, modularitu a celkovou stabilitu klientské části, včetně složitějších prvků uživatelského rozhraní, jako je technologie *Drag & Drop*.

Správnost a robustnost celého enginu byla úspěšně ověřena implementací tří odlišných herních pluginů – komplexní hry *Bang!* a jednodušších her *Prší* a *UNO*. Systém byl následně nasazen na školní server a testován s reálnými uživateli, což potvrdilo jeho funkčnost v síťovém provozu.

Projekt mi přinesl obrovské množství cenných zkušeností z oblasti full-stack vývoje, návrhových vzorů, řešení asynchronní komunikace a správy stavu aplikace. Vytvořená platforma má navíc potenciál do budoucna, kdy ji budu moci díky modularitě snadno rozšiřovat o nová rozšíření, přidávat zcela nové herní pluginy nebo se zaměřit na další vylepšování uživatelského rozhraní a vizuálních efektů klientské aplikace.

18 Přílohy

V odevzdávacím systému vyuka.gyarab.cz je přiložený repozitář projektu ke stavu k datu odevzdání projektu.

Reference

- [1] Dan Abramov. React dnd: Drag and drop for react. <https://react-dnd.github.io/react-dnd/about>.
- [2] Baeldung. A guide to org.json. <https://www.baeldung.com/java-org-json>.
- [3] Baeldung. List files in a directory in java. <https://www.baeldung.com/java-list-directory-files>.
- [4] Claudéric Demers. dnd kit: A modern drag and drop toolkit for react. <https://github.com/claudeiric/dnd-kit>.
- [5] Ecma International. The json data interchange syntax (standard ecma-404). <https://ecma-international.org/publications-and-standards/standards/ecma-404/>.
- [6] Dark gray wooden plank pattern. https://www.freepik.com/free-photo/dark-gray-wooden-plank-pattern_1037175.htm.
- [7] Google. Gemini ai, 2026. Analýza chybových hlášení, optimalizace dokumentace a konzultace Maven příkazů.
- [8] Google Inc. Gson api documentation. <https://www.javadoc.io/doc/com.google.code.gson/gson/latest/com.google.gson/com/google/gson/Gson.html>.
- [9] Interní systém avava. <https://avava.gyarab.cz/>.
- [10] Průvodce svs: Statický web. <https://guides.svs.gyarab.cz/staticky-web.html>.
- [11] ITnetwork. Latex tutoriál: Nadpisy a struktura. <https://www.itnetwork.cz/programovani/latex/latex-tutorial-nadpisy>.
- [12] Představení json. <https://www.json.org/json-cz.html>.
- [13] Timo Lins. React hot toast: Smoking hot notifications. <https://react-hot-toast.com/>.
- [14] Meta Platforms, Inc. React reference: usecontext. <https://react.dev/reference/react/useContext>, 2024.
- [15] Meta Platforms, Inc. React reference: useeffect. <https://react.dev/reference/react/useEffect>, 2024.
- [16] Meta Platforms, Inc. Rendering lists - keeping list items in order with key. <https://react.dev/learn/rendering-lists#keeping-list-items-in-order-with-key>, 2024.
- [17] Microsoft. Github copilot, 2026. Asistent při psaní kódu v TypeScriptu a Javě.
- [18] Bang! the bullet product page. <https://www.m-g.com.au/product/bang-the-bullet/>.
- [19] Mozilla Contributors. Html drag and drop api. https://developer.mozilla.org/en-US/docs/Web/API/HTML_Drag_and_Drop_API.
- [20] Mozilla Contributors. Json.parse: Bad parse errors. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Errors/JSON_bad_parse.
- [21] OpenAI. Chatgpt (verze gpt-4o a gpt-o1), 2026. Použito pro generování Python skriptů pro zpracování obrázků a konzultace herní logiky.
- [22] Oracle Corporation. How to write doc comments for the javadoc tool. <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>.

- [23] Martin Pulido. How to make a bang! character card. <https://bangcardgame.blogspot.com/2011/03/design-how-to-make-bang-character-card.html>.
- [24] Is there a way to create virtual ram in ubuntu? https://www.reddit.com/r/Ubuntu/comments/1gqneha/is_there_a_way_to_create_virtual_ram/.
- [25] Print and play version for bang! https://www.reddit.com/r/printandplay/comments/6p3ar6/print_and_play_version_for_bang/.
- [26] Do the json keys have to be surrounded by quotes? <https://stackoverflow.com/questions/949449/do-the-json-keys-have-to-be-surrounded-by-quotes>.
- [27] Java: Convert integer to string. <https://stackoverflow.com/questions/5071040/java-convert-integer-to-string>.
- [28] Java: Leaking "this" in constructor. <https://stackoverflow.com/questions/9851813/java-leaking-this-in-constructor>.
- [29] Random shuffling of an array. <https://stackoverflow.com/questions/1519736/random-shuffling-of-an-array>.
- [30] Rotate an image in html. <https://stackoverflow.com/questions/20061774/rotate-an-image-in-image-source-in-html>.
- [31] List subdirectories only n-level deep. <https://unix.stackexchange.com/questions/93323/list-subdirectories-only-n-level-deep>.
- [32] Vite: Building for production. <https://vite.dev/guide/build>.
- [33] Bang! (karetní hra). [https://cs.wikipedia.org/wiki/Bang!_\(karetn%C3%AD_hra\)](https://cs.wikipedia.org/wiki/Bang!_(karetn%C3%AD_hra)), 2025. Citováno: 27. 11. 2025.